چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۲

Fragment، FragmentManager و Navigation — در دنیای Compose



اسماعیل مرادی

@esmaeeil-moradi700

چرا Fragment در ۲۰۲۶ هنوز روز دوم است؟

صفحه‌های جدید معمولاً با Compose نوشته می‌شوند — ولی مصاحبه‌ها هنوز دربارهٔ Fragment می‌پرسند. چون:

- بیشتر اپ‌های واقعی هنوز **سال‌ها کد Fragment** دارند
- Fragment **منسوخ (deprecated) نیست** و AndroidX هنوز برایش قابلیت جدید می‌دهد (predictive back، هم‌کاری با Compose)
- کار واقعی در ۲۰۲۶ **هم‌کاری و مهاجرت** است: Compose داخل Fragment، و Fragment داخل Compose
- باگ‌های Fragment — نشت حافظه، از دست رفتن state، back stack — فیلتر کلاسیک مصاحبهٔ سینیور هستند

سطح	سؤال‌ها
جونیور	دو lifecycle، Fragment یا Activity یا Composable، ارتباط بین صفحه‌ها
مید	state و process death، تراکنش‌ها، back stack و دکمهٔ Back، تله‌های رایج
سینیور	طراحی Navigation، transitionهای روان، هم‌کاری با Compose و مهاجرت
آرکیتکت	Navigation چندتیمی، scope درست state، ANR و پاسخ‌گویی



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: دو lifecycle

Fragment **یک lifecycle برای خودش** دارد و **یک lifecycle جدا و کوتاه‌تر برای view**:

```
Fragment: CREATED ──── STARTED ─ RESUMED ─ … ─ DESTROYED
View:         onCreateView ──────── onDestroyView
              (view exists)         (view gone, Fragment
                                     may still be alive)
```

- Fragmentی که در back stack است **view ندارد**، اما هنوز زنده است (CREATED)
- view یک Fragment می‌تواند **بارها** از بین برود و دوباره ساخته شود
- هر چیزی که با view کار دارد باید از `viewLifecycleOwner` پیروی کند، نه از خود Fragment

> **قانونی که پشت نیمی از باگ‌های FRAGMENT است**
>
> Fragment از view خودش بیشتر عمر می‌کند. تک‌تک نشت‌ها و کرش‌های «view is null» در این اسلایدها به همین جمله برمی‌گردند.



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

دو lifecycle، Fragment یا Activity یا Composable، ارتباط بین صفحه‌ها



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. lifecycle فرگمنت و میزبانش

```
onAttach → onCreate → onCreateView → onViewCreated
→ onViewStateRestored → onStart → onResume
… onPause → onStop → onSaveInstanceState*
→ onDestroyView → onDestroy → onDetach
```

Callback	کاربرد
`onCreate`	خواندن `arguments` و گرفتن ViewModel. **هنوز view وجود ندارد**
`onCreateView`	inflate کردن layout یا برگرداندن `ComposeView` — نه هیچ کار دیگر
`onViewCreated`	bind کردن viewها و شروع collect با `viewLifecycleOwner`
`onDestroyView`	پاک کردن ارجاع به viewها (binding، adapterهایی که view نگه می‌دارند)
`onDestroy` / `onDetach`	نمونه از بین رفته / از میزبان جدا شده

* `onSaveInstanceState` ممکن است هر زمانی قبل از `onDestroy` اجرا شود — روی ترتیب مشخصی حساب نکنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. قواعدی که مصاحبه‌گر چک می‌کند

- state یک Fragment **هیچ‌وقت از state میزبانش جلوتر نمی‌رود**: اگر میزبان STARTED باشد، Fragmentهایش حداکثر STARTED هستند.
- `setMaxLifecycle` می‌تواند سقف را پایین‌تر بیاورد — `ViewPager2` صفحه‌های خارج از دید را در STARTED نگه می‌دارد و فقط صفحهٔ دیده‌شده RESUMED است.
- `onActivityCreated` **منسوخ شده**. برای راه‌اندازی view از `onViewCreated` استفاده کنید، یا lifecycle اکتیویتی را observe کنید.

تلهٔ مصاحبه

«*داده را از طریق constructor به Fragment بدهید.*» — **نه.** بعد از process death، سیستم Fragment را با **constructor بدون آرگومان** دوباره می‌سازد و آرگومان‌های constructor از دست می‌روند. از `arguments` (یا routeهای type-safe در Navigation) استفاده کنید. برای constructor injection به `FragmentFactory` نیاز دارید.

سؤال تکمیلی

«`viewLifecycleOwner` *چه زمانی در دسترس نیست؟*» — قبل از برگشتن `onCreateView` و بعد از `onDestroyView` . دسترسی به آن در این بازه‌ها exception می‌دهد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. Activity، Fragment یا Composable؟

انتخاب	چه زمانی
Activity	نقطه‌های ورود واقعی: launcher، deep link، share target، picture-in-picture، یک task جدا
Fragment	صفحه‌های View-based موجود، مهاجرت تدریجی، کتابخانه‌هایی که هنوز Fragment لازم دارند
مقصد Composable	صفحه‌های جدید در ۲۰۲۶ — یک Activity + Compose Navigation

تلهٔ مصاحبه

«*Fragment منسوخ شده.*» — **نشده.** گوگل برای UI جدید Compose را توصیه می‌کند، اما Fragment همچنان پشتیبانی می‌شود و قابلیت جدید می‌گیرد.

تلهٔ مصاحبه

«*از Fragment استفاده می‌کنیم چون Activity سنگین است.*» — جواب ضعیفی است. مزیت واقعی single-activity، **scope مشترک** (ViewModel، DI)، **یک گراف navigation** و **کنترل transitionها** است، نه صرفه‌جویی در حافظه.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. Fragmentها چطور با هم ارتباط می‌گیرند؟

نیاز	ابزار مدرن
state مشترک و پیوسته	**ViewModel مشترک** در scope درست: activity، گراف navigation یا Fragment والد
یک نتیجهٔ یک‌باره برای صفحهٔ قبل	**Fragment Result API**، یا `SavedStateHandle` مربوط به back-stack entry در Navigation
آرگومان برای صفحهٔ بعد	`arguments` / routeهای type-safe
صحبت با activity میزبان	ViewModel مشترک — از cast کردن `requireActivity()` به interface پرهیز کنید

```
// Receiver (e.g. ProfileFragment)
setFragmentResultListener(REQUEST_PICK_CITY) { _, bundle ->
    viewModel.onCitySelected(bundle.getString(KEY_CITY))
}

// Sender (e.g. CityPickerFragment)
setFragmentResult(REQUEST_PICK_CITY, bundleOf(KEY_CITY to city))
findNavController().popBackStack()
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. جزئیاتی که فرق می‌سازند

- نتیجه‌ها فقط وقتی تحویل داده می‌شوند که listener **حداقل STARTED** باشد، و برای هر key فقط **آخرین** نتیجه نگه داشته می‌شود.
- نتیجه‌ها از configuration change و process death جان سالم به در می‌برند — بخشی از saved state خود `FragmentManager` هستند.
- با Navigation:

```
// Sender
findNavController().previousBackStackEntry
    ?.savedStateHandle?.set(KEY_CITY, city)

// Receiver
findNavController().currentBackStackEntry
    ?.savedStateHandle?.getStateFlow<String?>(KEY_CITY, null)
```

تلهٔ مصاحبه

هیچ‌وقت ارجاع مستقیم به یک Fragment دیگر نگه ندارید (مثلاً `findFragmentByTag(...)` و بعد صدا زدن متدهایش). ممکن است از بین رفته باشد، دوباره ساخته شده باشد یا view نداشته باشد — و دو صفحه را برای همیشه به هم گره می‌زند.



اسماعیل مرادی
@esmaeeil-moradi700

مید

state و process death، تراکنش‌ها، back stack، تله‌های رایج



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. state فرگمنت: چرخش صفحه و process death

نوع state	کجا نگه داشته می‌شود
آرگومان‌های صفحه	Bundle مربوط به `arguments` — که مقدار پیش‌فرض `SavedStateHandle` در ViewModel هم هست
کلیدهای لازم برای بازسازی صفحه	`SavedStateHandle`
state خود view (متن، اسکرول، تیک)	برای viewهای **دارای ID** خودکار ذخیره می‌شود
دادهٔ صفحه	ViewModel (حافظه) + repository (دیسک / شبکه)

- `setRetainInstance(true)` **منسوخ شده** — ViewModel همان نیاز را بدون نگه داشتن خود Fragment برآورده می‌کند.
- خود back stack — و saved state همهٔ Fragmentهای داخلش — بعد از process death توسط `FragmentManager` بازگردانده می‌شود.

تلهٔ مصاحبه

«*saved state فرگمنت بعد از process death باقی نمی‌ماند.*» — **غلط است**؛ درست مثل Activity: در process death که سیستم انجام می‌دهد باقی می‌ماند، و فقط وقتی کاربر خودش task را ببندد از بین می‌رود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. درست تستش کنید

روش	چه چیزی را شبیه‌سازی می‌کند
`FragmentScenario.recreate()`	configuration change
اپ را به background ببرید، `adb shell am kill <package>` بزنید، بعد از Recents بازش کنید	**process death**
گزینهٔ «Don't keep activities»	از بین رفتن Activity — **پروسه زنده می‌ماند**

```kotlin
@Test fun keepsQueryAfterRecreate() {
    val scenario = launchFragmentInContainer<SearchFragment>()
    onView(withId(R.id.query)).perform(replaceText("kotlin"))

    scenario.recreate()

    onView(withId(R.id.query)).check(matches(withText("kotlin")))
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. تراکنش‌ها: هر عملیات دقیقاً چه می‌کند

عملیات	Fragment قبلی	view آن
`add(B)` روی A	A همچنان **RESUMED** می‌ماند	view مربوط به A (زیر B) می‌ماند
`replace(B)` با back stack	A به **CREATED** می‌رود	از بین می‌رود (`onDestroyView`)
`replace(B)` بدون back stack	A به **DESTROYED** می‌رود	از بین می‌رود
`hide(A)` / `show(A)`	lifecycle **تغییر نمی‌کند**	`GONE` / نمایان
`detach(A)` / `attach(A)`	CREATED، هنوز مدیریت‌شده	از بین می‌رود / دوباره ساخته می‌شود

تلهٔ مصاحبه

«`add()` *فرگمنت زیری را pause می‌کند.*» — **نه.** هر دو Fragment در حالت RESUMED هستند، هر دو کلیک می‌گیرند مگر اینکه بالایی آن را مصرف کند، و هر دو viewشان را در حافظه نگه می‌دارند. برای همین `replace` گزینهٔ پیش‌فرض است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. commit امن

```
supportFragmentManager.commit {
    setReorderingAllowed(true)
    replace<DetailFragment>(
        R.id.container, args = bundleOf("id" to id)
    )
    addToBackStack("detail")
}
```

- همیشه `setReorderingAllowed(true)` — برای transitionهای postponed لازم است و از کار اضافهٔ lifecycle جلوگیری می‌کند.
- `commit()` **ناهمگام (asynchronous)** است؛ `commitNow()` همگام است ولی **نمی‌شود** آن را به back stack اضافه کرد.
- میزبان را `FragmentContainerView` بگذارید، نه `FrameLayout`: ترتیب z انیمیشن‌ها را درست می‌کند و window insets را پخش می‌کند.

تلهٔ مصاحبه

commit بعد از اینکه میزبان state خود را ذخیره کرده `IllegalStateException` می‌دهد. `commitAllowingStateLoss()` کرش را پنهان می‌کند، چون اگر پروسه بمیرد آن تراکنش را **بی‌صدا دور می‌اندازد**. به‌جایش زمان‌بندی را درست کنید: فقط وقتی lifecycle حداقل STARTED است commit کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. back stack و مدیریت دکمهٔ Back

- `addToBackStack(name)` **تراکنش** را ثبت می‌کند؛ Back آن را برعکس اجرا می‌کند.
- `popBackStack(name, POP_BACK_STACK_INCLUSIVE)` تا یک entry نام‌دار (و خود آن) را pop می‌کند.
- **چند back stack** (bottom navigation): `saveBackStack(name)` / `restoreBackStack(name)` — برای Navigation / `saveState` / `restoreState` از همین‌ها استفاده می‌کند.

```
// Enabled only while there is something to intercept
val discardCallback = requireActivity().onBackPressedDispatcher
    .addCallback(viewLifecycleOwner, enabled = false) {
        showDiscardDialog()
    }

viewLifecycleOwner.lifecycleScope.launch {
    viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
        viewModel.hasUnsavedChanges.collect { unsaved ->
            discardCallback.isEnabled = unsaved
        }
    }
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. دکمهٔ Back در ۲۰۲۶: predictive back

ANDROID 16 (API 36)

در اپ‌هایی که API 36 را target می‌کنند، انیمیشن predictive back به‌صورت پیش‌فرض روشن است و `Activity.onBackPressed()` **دیگر صدا زده نمی‌شود**؛ `KEYCODE_BACK` هم dispatch نمی‌شود. منطق Back باید در callbackهای `OnBackPressedDispatcher` باشد (یا `BackHandler` در Compose).

- Fragment نسخهٔ 1.7 به بعد، predictive back را برای transitionهای Fragment با Animator و AndroidX Transitions پشتیبانی می‌کند.
- callback باید **فقط وقتی فعال باشد که واقعاً Back را مدیریت می‌کند** — callbackی که همیشه فعال است انیمیشن predictive را خراب می‌کند، چون سیستم نمی‌تواند بفهمد Back از صفحه خارج می‌شود.

تلهٔ مصاحبه

«*`onBackPressed()` را در activity override می‌کنم و به Fragment فعلی پاس می‌دهم.*» — از API 33 منسوخ شده و با targetSdk 36 نادیده گرفته می‌شود. از callbackهای dispatcher که فعال/غیرفعال می‌شوند استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. تله‌های رایج — و راه‌حل

تله	راه‌حل
نگه داشتن binding بعد از `onDestroyView` ← نشت حافظه	در `onDestroyView` مقدار `_binding = null` بگذارید
collect کردن با lifecycle خود Fragment	`viewLifecycleOwner` + `repeatOnLifecycle`
`requireContext()` در callback بعد از detach ← کرش	کار را به lifecycle ویو/فرگمنت محدود کنید؛ در stop لغوش کنید
Fragmentهای تودرتو روی manager اشتباه	فرزندها: `childFragmentManager`؛ هم‌سطح‌ها: `parentFragmentManager`
نگه داشتن ارجاع Fragment در adapter مربوط به `ViewPager2`	بگذارید `FragmentStateAdapter` آن‌ها را بسازد؛ ارتباط از طریق ViewModel
adapter مربوط به RecyclerView که بیشتر از view عمر می‌کند	اگر adapter یک property است، در `onDestroyView` مقدار `recyclerView.adapter` را پاک کنید



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. Fragment امن استاندارد

```
class OrdersFragment : Fragment(R.layout.fragment_orders) {

    private val viewModel: OrdersViewModel by viewModels()
    private var _binding: FragmentOrdersBinding? = null
    private val binding get() = checkNotNull(_binding)

    override fun onViewCreated(view: View, state: Bundle?) {
        _binding = FragmentOrdersBinding.bind(view)
        val owner = viewLifecycleOwner
        owner.lifecycleScope.launch {
            owner.repeatOnLifecycle(Lifecycle.State.STARTED) {
                viewModel.uiState.collect(::render)
            }
        }
    }

    override fun onDestroyView() {
        _binding = null
        super.onDestroyView()
    }
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

طراحی Navigation، transitionهای روان، هم‌کاری با Compose و مهاجرت



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. Navigation برای اپ single-activity مبتنی بر Fragment

- **Jetpack Navigation** با یک `NavHostFragment` داخل `FragmentContainerView`
- **routeهای type-safe** (`@Serializable`) با Kotlin DSL — جایگزین مدرن XML + Safe Args
- **گراف تودرتو** برای هر flow (onboarding، checkout) — که scope طبیعی ViewModelهای آن flow است
- **deep link روی خود مقصدها** تعریف شود تا لینک، back stack درست را بسازد
- UI با یک **interface کوچک navigator** حرف بزند، نه مستقیم با `NavController` — تستش راحت‌تر است و انتقال به Compose هم ساده‌تر

```
navController.graph = navController.createGraph(
    startDestination = Home
) {
    fragment<HomeFragment, Home> { label = "Home" }
    fragment<DetailFragment, Detail> {
        deepLink<Detail>(basePath = "https://example.com/item")
    }
}
// Navigate
findNavController().navigate(Detail(itemId = 42))
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. آرگومان‌ها، نتیجه‌ها و تست

```
@Serializable data object Home
@Serializable data class Detail(val itemId: Long)

class DetailViewModel(handle: SavedStateHandle) : ViewModel() {
    private val itemId = handle.toRoute<Detail>().itemId
}
```

- آرگومان‌ها در زمان کامپایل چک می‌شوند و در saved state مربوط به back-stack entry نگه داشته می‌شوند ← از process death جان سالم به در می‌برند.
- نتیجه‌ها: `SavedStateHandle` مربوط به back-stack entry قبلی (سؤال ۳).
- تست navigation با `TestNavHostController` — روی `currentBackStackEntry` assert کنید، بدون نیاز به UI دستگاه.

تلهٔ مصاحبه

navigate دوباره با دو ضربهٔ سریع، کرش می‌دهد یا صفحه را تکراری باز می‌کند. با چک کردن مقصد فعلی جلویش را بگیرید، یا بگذارید ViewModel ناوبری را به‌صورت stateی ارائه دهد که فقط یک بار مصرف می‌شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. تراکنش‌ها و transitionهای روان

ترد اصلی را در طول transition آزاد نگه دارید

- در `onCreateView` / `onViewCreated` داده load نکنید — ViewModel داده را می‌آورد؛ Fragment فقط state را رندر می‌کند
- از داخل Fragment کار `Dispatchers.IO` راه نیندازید که بعد به viewها دست بزند — repository باید توابع `suspend` امن برای ترد اصلی (main-safe) بدهد

Transitionها

- همه‌جا `setReorderingAllowed(true)`
- `postponeEnterTransition()` تا وقتی تصویرها یا لیست آماده شوند، بعد `startPostponedEnterTransition()` — با timeout، تا یک load ناموفق navigation را قفل نکند
- برای transitionهای یکدست و سازگار با GPU از Material motion استفاده کنید: `MaterialSharedAxis` و `MaterialContainerTransform`

لیست‌ها

- `ListAdapter` (مقایسهٔ diff روی ترد پس‌زمینه) و stable ID هر جا که کمک کند
- `setHasFixedSize(true)` فقط وقتی تغییر آیتم‌ها هیچ‌وقت اندازهٔ خود RecyclerView را عوض نمی‌کند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. اندازه بگیرید، حدس نزنید

```
override fun onViewCreated(view: View, savedInstanceState: Bundle?)
{
    postponeEnterTransition(500, TimeUnit.MILLISECONDS)
    binding.list.doOnPreDraw { startPostponedEnterTransition() }
}
```

- **JankStats** در production، با برچسب صفحهٔ فعلی
- **Macrobenchmark** با `FrameTimingMetric` برای سناریوهای «باز کردن جزئیات» و «اسکرول لیست» در CI
- **Perfetto** برای پیدا کردن فریم‌های طولانی: inflate، layout، bind در `RecyclerView`
- **StrictMode** در نسخهٔ debug برای گرفتن دسترسی به دیسک و شبکه روی ترد اصلی



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. هم‌کاری Compose و Fragment

Compose داخل composition — **Fragment** را با *view* از بین ببرید، نه با window:

```
override fun onCreateView(
    inflater: LayoutInflater, container: ViewGroup?, state: Bundle?,
): View = ComposeView(requireContext()).apply {
    setViewCompositionStrategy(
        ViewCompositionStrategy.DisposeOnViewTreeLifecycleDestroyed
    )
    setContent { AppTheme { ProfileRoute(viewModel) } }
}
```

Fragment داخل Compose — با `fragment-compose`:

```
@Composable
fun MapPane(modifier: Modifier = Modifier) {
    AndroidFragment<LegacyMapFragment>(modifier.fillMaxSize())
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. یک استراتژی مهاجرت که جواب می‌دهد

۱. **اول theme:** یک theme در Compose که با theme ویوها یکی باشد.

۲. **UI برگ‌ها:** کامپوننت‌ها و آیتم‌های لیست جدید با Compose، داخل Fragmentهای موجود از طریق `ComposeView`.

۳. **صفحه به صفحه:** Fragment تبدیل به یک پوستهٔ نازک دور یک صفحهٔ composable می‌شود؛ ViewModelها دست نمی‌خورند.

۴. **Navigation در آخر:** وقتی بیشتر صفحه‌ها composable شدند، گراف Fragment را با Compose Navigation عوض کنید. صفحه‌های قدیمی را تا بازنویسی با `AndroidFragment` نگه دارید.

تلهٔ مصاحبه

استراتژی پیش‌فرض، composition را وقتی view **از window جدا می‌شود** از بین می‌برد. view یک Fragment ممکن است بدون اینکه destroy شود جدا و دوباره وصل شود (در transitionها و بعضی pagerها)، و state مربوط به `remember` از دست می‌رود. `DisposeOnViewTreeLifecycleDestroyed` عمر composition را به **lifecycle ویوی Fragment** گره می‌زند.

سؤال تکمیلی

«*چرا همه‌چیز را یک‌جا بازنویسی نکنیم؟*» — به‌خاطر ریسک و سرعت. مهاجرت تدریجی اجازه می‌دهد فیچر تحویل دادن ادامه پیدا کند و هر قدم جدا تست و در صورت نیاز rollback شود.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

Navigation چندتیمی، scope درست state، ANR و پاسخ‌گویی



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. Navigation بین چند تیم و چند ماژول



- هر فیچر **routeها و نقطه‌های ورودش** را در یک ماژول کوچک `:api` منتشر می‌کند؛ UI در `:impl` است.
- فیچرها فقط به `:api` فیچرهای دیگر وابسته‌اند ← بدون وابستگی حلقوی، با build مستقل.
- هر `:impl` **گراف خودش را** از طریق DI multibinding به اپ **اضافه می‌کند**؛ `:app` گراف ریشه را سرهم می‌کند.
- deep linkها روی مقصدهای داخل هر فیچر تعریف و در CI اعتبارسنجی می‌شوند.
- dynamic feature module فقط وقتی حجم نصب واقعاً مجبورتان کند — build، تست و navigation را پیچیده‌تر می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. اضافه کردن گراف‌ها با DI

```
// :feature:checkout:api
@Serializable data object CheckoutGraph

// :core:navigation
interface FeatureGraph {
    fun NavGraphBuilder.register()
}

// :feature:checkout:impl
class CheckoutFeatureGraph @Inject constructor() : FeatureGraph {
    override fun NavGraphBuilder.register() {
        navigation<CheckoutGraph>(startDestination = Cart) {
            fragment<CartFragment, Cart> { label = "Cart" }
            fragment<PaymentFragment, Payment> { label = "Payment" }
        }
    }
}

@Module @InstallIn(SingletonComponent::class)
interface CheckoutNavModule {
    @Binds @IntoSet
    fun graph(impl: CheckoutFeatureGraph): FeatureGraph
}
```

`app:` گراف ریشه را با صدا زدن `register()` روی همهٔ `FeatureGraph`های موجود در `Set` تزریق‌شده می‌سازد. `FragmentFactory` هم constructor injection را برای Fragmentها ممکن می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. اشتراک state در سلسله‌مراتب عمیق

مالک ViewModel را با این سؤال انتخاب کنید: *«این state باید با چه چیزی زنده بماند و با چه چیزی بمیرد؟»*

Scope	تا کی زنده است	state معمول
Fragment (`viewModels()`)	تا destroy شدن Fragment	یک صفحه
Fragment والد	تا destroy شدن والد	تب‌ها یا پنل‌های داخل یک صفحه
گراف navigation تودرتو	تا pop شدن flow از back stack	checkout چندمرحله‌ای، onboarding
Activity (`activityViewModels()`)	تا finish شدن Activity	state واقعاً سراسری UI — نادر

```
// Tabs share their parent's ViewModel
private val pane: PaneViewModel by viewModels(
    ownerProducer = { requireParentFragment() }
)

// Every step of checkout shares the flow's ViewModel
private val checkout: CheckoutViewModel by viewModels(
    ownerProducer = {
        findNavController().getBackStackEntry<CheckoutGraph>()
    }
)
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. مقیاس‌پذیر نگه داشتن state مشترک

- **باریک‌ترین scope که کار می‌کند.** یک «ViewModel خدا» با scope اکتیویتی، آهنربای merge conflict و یک متغیر سراسری پنهان می‌شود.
- **ViewModel مشترک state نگه می‌دارد، نه منطق UI.** هر صفحه ViewModel خودش را دارد و از ViewModel مشترک می‌خواند.
- **eventها را به شکل state مدل کنید.** یک event یک‌باره در `SharedFlow` اگر کسی collect نکند از دست می‌رود؛ stateی (مثل `paymentResult: Result?`) که UI مصرف و پاکش می‌کند چیزی را گم نمی‌کند.
- **چیزهای مهم را پایدار کنید.** state یک flow که باید از process death عبور کند به `SavedStateHandle` (کلیدها) یا repository (داده) می‌رود.

تلهٔ مصاحبه

scope دادن به activity «چون راحت است» باعث نشت state یک flow به صفحه‌های بی‌ربط می‌شود: checkout را تمام کنید، دوباره شروع کنید — و سبد خرید قبلی هنوز آنجاست.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. lifecycle فرگمنت، ANR و پاسخ‌گویی

همهٔ callbackهای lifecycle روی **ترد اصلی** اجرا می‌شوند. ANR وقتی رخ می‌دهد که این ترد بلاک شود:

عامل ANR	سقف زمانی
رویداد ورودی (input) پردازش نشود	۵ ثانیه
`BroadcastReceiver.onReceive`	۱۰ ثانیه در پیش‌زمینه / ۶۰ ثانیه در پس‌زمینه

آستانهٔ «رفتار بد» در Android vitals: **نرخ ANR درک‌شده توسط کاربر ۰٫۴۷ درصد** در کل، و **۸ درصد** روی هر مدل دستگاه.

- کار ترد اصلی در هر callback باید در حد **میلی‌ثانیه** باشد: inflate، bind، شروع collect.
- دیسک، شبکه، parse کردن JSON، رمزنگاری، decode تصویر ← بیرون از ترد اصلی، پشت APIهای `suspend` امن.
- فریم‌های janky (بیش از ۱۶ میلی‌ثانیه در ۶۰Hz، بیش از ۸ میلی‌ثانیه در ۱۲۰Hz) خیلی قبل از ANR ظاهر می‌شوند — اندازه‌شان بگیرید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. پیدا کردن ANR در production

```
val am = context.getSystemService(ActivityManager::class.java)
am.getHistoricalProcessExitReasons(context.packageName, 0, 5)
    .filter { it.reason == ApplicationExitInfo.REASON_ANR }
    .forEach { exit ->
        val trace = exit.traceInputStream
            ?.bufferedReader()?.use { it.readText() }
        crashReporter.reportAnr(
            exit.timestamp, exit.description, trace
        )
    }
```

- `ApplicationExitInfo` (از API 30) در اجرای بعدی، **trace مربوط به ANR را از دستگاه خود کاربر** می‌دهد.
- StrictMode در debug، Android vitals در production، و Perfetto برای ریشه‌یابی.
- مقصرهای معمول: `SharedPreferences` با `commit()` بلاک می‌کند، و حتی نوشتن‌های معلق `apply()` هم در `onStop` روی ترد اصلی flush می‌شوند — به DataStore مهاجرت کنید)، کوئری دیتابیس در `onViewCreated`، و layoutهای سنگینی که با هر تعویض تب inflate می‌شوند.

جواب سینیور در یک جمله

«lifecycle به شما می‌گوید کِی می‌توانید به UI دست بزنید — نه اینکه کار را *کجا* انجام دهید. جای کار در ViewModel و لایهٔ داده است، بیرون از ترد اصلی.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۲ — خلاصهٔ یک‌صفحه‌ای

- Fragment **از view خودش بیشتر عمر می‌کند** — برای هر چیزی که با view کار دارد از `viewLifecycleOwner` استفاده کنید.
- binding را در `onDestroyView` پاک کنید. آرگومان در constructor نه — از `arguments` استفاده کنید.
- ارتباط از طریق ViewModelهای با scope درست، Fragment Result API یا نتیجه‌های navigation — هرگز ارجاع مستقیم.
- `add()` فرگمنت زیری را RESUMED نگه می‌دارد؛ `replace` + back stack را ترجیح دهید.
- `setReorderingAllowed(true)`، `FragmentContainerView`، و بدون وصله‌پینهٔ `commitAllowingStateLoss`.
- targetSdk 36: متد `onBackPressed()` صدا زده نمی‌شود — از `OnBackPressedDispatcher` استفاده کنید.
- routeهای type-safe؛ deep link روی مقصدها؛ state هر flow در scope گراف تودرتو.
- Compose در Fragment: `DisposeOnViewTreeLifecycleDestroyed`. و Fragment در Compose: `AndroidFragment`.
- lifecycle می‌گوید *کِی*، نه *کجا* — کار سنگین هیچ‌وقت روی ترد اصلی اجرا نمی‌شود.

روز ۳ در راه است ← این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700